

Test Plan

OpenCart Website (opencart.com) — Registration, Login & Account Details Modules

Application Under Test	https://www.opencart.com/
Document Version	v1.1
Author	Gohar Zaman (QA Engineer)
Last Updated	16 July 2026
Status	Approved for execution
Reference Documents	Test_Cases-OpenCartWebsite.xlsx TestRail Execution Report.pdf Bug_Report-OpenCartWebsite.xlsx Test Summary Report-OpenCart.pdf

Revision History

Version	Date	Author	Description of Change
v1.0	14 July 2026	Gohar Zaman	Initial draft covering the three assigned modules
v1.1	16 July 2026	Gohar Zaman	Email verification brought into scope (inbox access confirmed); table of contents added

Table of Contents

1. Overview

1.1 Purpose of This Test Cycle

1.2 Objectives

2. Test Scope

2.1 Modules Under Test

2.2 Exclusions

3. Testing Approach

4. Test Environment

5. Coverage Summary

6. Test Data Strategy

7. Resources & Effort

7.1 Team

7.2 Effort Breakdown

8. Entry & Exit Criteria

9. Risks & Assumptions

10. Deliverables

11. Execution Sequence

1. Overview

1.1 Purpose of This Test Cycle

opencart.com is the official website of the OpenCart platform. Visitors register a single account on the site to buy or publish marketplace extensions, download the software, and raise support requests. This document lays out how QA will verify the three account-related modules assigned for this cycle, Registration, Login, and Account Details, including the transactional emails the site sends for account creation and password recovery, since a monitored test inbox is available.

Step-level test cases live in TestRail rather than in this document; exports of the case repository and the executed run accompany this plan as reference material. The sections below describe the boundaries of testing, the techniques applied, the environment, effort, and the conditions for starting and closing the cycle.

1.2 Objectives

- Confirm that account creation works end to end, form validations fire correctly, a new account is established, and the welcome/thank-you email arrives at the registered address.
- Confirm that authentication behaves as expected for both good and bad credentials, and that password recovery can be completed fully through the emailed reset link.
- Confirm that a signed-in user can review and modify their profile information, change their password, and manage their newsletter preference, with every change persisting correctly.
- Surface and log any functional, validation, or usability defects found within these modules.

2. Test Scope

2.1 Modules Under Test

#	Module	What Will Be Verified
1	Registration	All fields on the sign-up form and their validation rules, mandatory-field enforcement, privacy policy acceptance, newsletter opt-in, the post-registration success state, behaviour when an email address is already taken, and the arrival plus content of the registration confirmation email
2	Login	Sign-in with correct and incorrect credentials, clarity of error messages, the complete forgotten-password journey (request, reset email arrival, link validity, setting a new password, signing in with it), session handling on logout and browser back-navigation, and throttling/lockout after repeated failed attempts
3	Account Details	Navigation within the account area, editing profile fields (name, email address, and any other editable attributes), validation on edited values, the password-change flow, newsletter subscribe and unsubscribe, and confirmation that saved changes survive a fresh login

2.2 Exclusions

- Marketplace features, browsing, purchasing, publishing, or downloading extensions

- Community forum, blog, documentation pages, demo store, support ticketing, and OpenCart Cloud sign-up or billing
- Load, stress, and formal penetration testing, although malformed and injection-style inputs are still exercised as negative cases on the in-scope forms
- Mail-server infrastructure concerns such as spam scoring or deliverability across different email providers; what is verified is that the expected emails reach the monitored test inbox with correct content

3. Testing Approach

- **Design techniques:** boundary value analysis and equivalence classes drive the field-level cases; negative scenarios come from error guessing; the email and password-recovery journeys are covered through scenario-based, end-to-end cases.
- **Test types:** positive and negative functional checks, input validation, email content verification, UI and usability review, a cross-browser sanity sweep, plus a short exploratory session on each module.
- **Execution:** the cycle is manual. Every case is executed against a TestRail run, and the exported run results form part of the closing deliverables. Cases are written in an automation-friendly style so they can later be scripted in Playwright or Selenium without rework.
- **Defect handling:** each defect is raised with reproduction steps, expected versus actual outcome, severity, the environment it was seen on, and supporting screenshots or email captures.

4. Test Environment

Item	Detail
Application URL	https://www.opencart.com/
Primary browser	Chrome (latest stable)
Sanity browsers	Firefox, Edge and Safari — latest versions, high-value cases only
Devices / viewports	Desktop at 1920×1080; responsive layout spot-checked on a mobile viewport
Email verification	Monitored test mailboxes with full inbox access, used to validate registration and password-reset emails

5. Coverage Summary

The TestRail repository covers, per module:

- **Registration** — successful sign-up including the confirmation email (recipient, subject, body), each field's mandatory and format rules, minimum/maximum lengths on names and password, mismatched password confirmation, an already-registered email address, leading/trailing whitespace, script and SQL-style payloads, privacy-policy enforcement, newsletter opt-in state, and visual/accessibility checks on the form.
- **Login** — correct credentials, wrong password, unknown email (checking the site does not reveal which accounts exist), blank submissions, password case handling, email case handling, the full

recovery path from 'Forgotten Password' through the emailed link to a successful sign-in with the new password (and rejection of the old one), lockout on repeated failures, injection payloads, logout followed by back-button navigation, and page-level UI review.

- **Account Details** — pre-population of existing profile data, legitimate edits to name and email with persistence confirmed after logging out and back in, switching the account email to an address already in use, blank or malformed values on edit, password change (happy path, wrong current password, mismatched confirmation, boundary lengths) with old-versus-new password login checks, newsletter toggle in both directions, abandoning an edit without saving, hostile input strings, and UI review of the account pages.

6. Test Data Strategy

Data Item	Approach
Primary account	Registered fresh at the start of each session using a monitored test mailbox
Duplicate-email case	Re-attempt registration with the address created earlier in the same session
Alternate email	A second monitored mailbox, reserved for the change-of-email scenario
Profile edit values	A distinct set of replacement names/values so persisted changes are unambiguous
Boundary inputs	Name and password strings at documented min/max lengths; deliberately oversized strings for overflow checks

7. Resources & Effort

7.1 Team

Role	Count	Responsibilities
QA Engineer	1	Owns the cycle end to end: case review, data preparation, execution across all three modules, email verification, cross-browser sweep, exploratory sessions, defect logging and final reporting

7.2 Effort Breakdown

The cycle is sized at roughly 11 hours for one QA engineer:

Activity	Hours
Plan and case authoring, TestRail setup, data preparation	3.5
Registration execution (incl. email checks)	1.5
Login execution (incl. full password-recovery flow)	1.0
Account Details execution	2.0

Cross-browser sanity sweep	0.5
Exploratory sessions (timeboxed, one per module)	1.0
Defect logging and evidence capture	1.0
Test summary report	0.5
Total	11.0

8. Entry & Exit Criteria

8.1 Entry

- The site is reachable and stable, and the test mailboxes are receiving mail.
- Test cases have been reviewed in TestRail and all test data is ready.

8.2 Exit

- Every planned case has been executed and its result recorded.
- All Blocker and High severity defects are logged, and fixed ones re-verified where applicable.
- The summary report — pass/fail counts and the open defect list — has been issued.

9. Risks & Assumptions

#	Risk / Assumption	How It Is Handled
1	The site is live production; accounts created for testing remain on the system and cannot be removed by QA	Test accounts use clearly identifiable names and harmless data, and edits are confined strictly to accounts QA created
2	Transactional emails can arrive late or be filtered to junk, which could look like a failure	Wait a sensible interval and inspect the junk folder before raising a defect; re-trigger once to confirm the behaviour
3	No formal requirements exist — expected behaviour is inferred from standard account-management conventions and OpenCart's own patterns	Anything ambiguous is recorded as an observation for discussion instead of being logged straight as a defect
4	As a public production site, opencart.com may apply CAPTCHA, rate limits or bot protection that interferes with repeated negative attempts	Pace repeated attempts, and re-check any intermittent failure before treating it as a genuine defect

10. Deliverables

- This test plan (current version as per the revision history)
- Test case workbook, imported into TestRail
- Screenshots evidencing the TestRail case repository
- Execution report exported from TestRail at cycle close

- Bug report workbook with reproduction steps, severities and screenshots
- Closing test summary report

11. Execution Sequence

1. Registration — runs first, since it creates the accounts every later module relies on, and its email checks confirm the mailboxes are working.
2. Login — including the complete password-recovery journey.
3. Account Details — needs an authenticated session from the earlier modules.
4. Cross-browser sanity sweeps over the highest-value cases.
5. A timeboxed exploratory session (about 45 minutes) on each module to catch anything scripted coverage missed.